

LovdogModelSelector

Automated ML Framework

Contents

1	Data Management Core (LovdogDF.py)	3
1.1	Main Class: LovdogDataFrames	3
1.2	JSON Configuration System	3
1.3	Data Loading and Processing	4
1.3.1	JSON Parsing and Data Loading	4
1.4	Data Balancing Methods	5
1.4.1	Class Balancing	5
1.4.2	Undersampling Implementation	5
1.4.3	Oversampling Implementation	6
1.5	Data Access Methods	7
2	Data Loading Utilities (LovdogData.py)	9
2.1	Main Loading Function	9
2.2	2D Array Loading	9
2.3	2D Row-wise Loading	10
2.4	Data Augmentation	11
3	Feature Extraction (LovdogLBP.py)	13
3.1	Main Class: LBPFeatures	13
3.2	LBP Computation	13
3.3	Feature Extraction from LBP	14
3.4	Window-based Feature Analysis	15
3.5	GLCM Feature Extraction	16
3.6	Machine Learning Data Preparation	16
4	Model Selection (LovdogMS.py)	19
4.1	Main Class: ModelSelection	19
4.2	Default Classifier Configuration	19
4.3	Core Model Selection Process	20
4.4	Comprehensive Performance Metrics	21
4.5	Best Model Selection	22
4.6	Diagnostic Visualization	22
4.7	Model Persistence and Export	23

Chapter 1

Data Management Core (LovdogDF.py)

The **LovdogDataFrames** class is the central data management system that orchestrates all data loading, organization, and preprocessing. It serves as the foundation upon which feature extraction and model selection are built.

1.1 Main Class: LovdogDataFrames

```
1 class LovdogDataFrames:
2     def __init__(self, json_file_path=None,
3                   loader_function=None):
4         self.json_file_path = json_file_path
5         self.loader_function = loader_function
6         self.json_data = None
7         self.frames = {}
8         self.class_names = []
9
10        if json_file_path:
11            self.load_json(json_file_path)
```

Explanation: The constructor initializes the data management system with two key parameters:

- `json_file_path`: Path to a JSON configuration file that defines your data sources, classes, and loading parameters
- `loader_function`: A custom function that handles the actual data loading from files (defaults to a simple path returner)

The class maintains several important data structures:

- `frames`: A dictionary where keys are class names and values are lists of loaded data (images, arrays, etc.)
- `class_names`: List of all unique class labels found in the data
- `extra_data`: Stores additional metadata for each class

1.2 JSON Configuration System

```
1 {
2     "csv": "Features_Table.csv",
3     "class1": {
4         "class": "positive",
5         "path": ["data/class1/"],
6         "extra": {
7             "cols": 100,
8             "rows": 100,
9             "type": "2d"
10        }
11    },
12    "class2": {
13        "class": "negative",
14        "path": ["data/class2/"],
15        "extra": {
16            "cols": 100,
```

```

17         "rows": 100,
18         "type": "2d"
19     }
20 }
21 }

```

Explanation: The JSON configuration provides a declarative way to define your entire dataset:

- csv: Optional name for the output features table
- Each class definition contains:
 - class: The label for this category
 - path: List of directories or files containing data
 - extra: Loading parameters specific to your data format

The extra parameters are passed to the loader function and typically include:

- cols, rows: Dimensions for reshaping 1D data to 2D
- type: Data format ('2d', '2drow')
- delimiter: For text file parsing

1.3 Data Loading and Processing

1.3.1 JSON Parsing and Data Loading

```

1 def _parse_json(self):
2     """Internal method to parse the loaded JSON data using the provided loader function."""
3     if not self.json_data:
4         raise ValueError("No JSON data loaded. Call load_json() first.")
5
6     # Reset state
7     self.frames = {}
8     self.extra_data = {}
9     self.class_names = []
10
11     # Get the CSV name
12     self.feature_table_name = self.json_data.get("csv", "Features_Table.csv")
13
14     # Iterate through the JSON items
15     for key, value in self.json_data.items():
16         if key == "csv":
17             continue
18
19         if isinstance(value, dict):
20             class_name = value.get("class", f"Unknown_Class_{key}")
21             paths = value.get("path", [])
22             extra = value.get("extra", {})
23
24             self.extra_data[class_name] = extra
25             self.class_names.append(class_name)
26             self.frames[class_name] = [] # Initialize the list for this class
27
28             print(f"[INFO] Loading data for class: {class_name}")
29             for path in paths:
30                 path_obj = Path(path)
31                 if path_obj.is_file():
32                     # USE THE PROVIDED LOADER FUNCTION HERE
33                     loaded_data = self.loader_function(path_obj, extra)
34                     self._add_loaded_data(loaded_data, class_name)

```

Explanation: This method is the workhorse that processes the JSON configuration:

1. **Resets internal state** to ensure clean loading
2. **Extracts CSV name** for future feature table export
3. **Iterates through each class definition** in the JSON

4. **Processes each path** for the class (files or directories)
5. **Uses the loader function** to actually load the data
6. **Organizes data by class** in the frames dictionary

The method handles both individual files and directories recursively, making it flexible for various data organization schemes.

1.4 Data Balancing Methods

1.4.1 Class Balancing

```

1 def balance_frames(self, method='undersample', random_state=None):
2     """
3     Balance the frames at the data loading level (before feature extraction).
4
5     Parameters:
6     -----
7     method : str, default='undersample'
8         Method to balance classes: 'undersample' or 'oversample'
9
10    random_state : int, optional
11        Random seed for reproducibility
12    """
13    if not self.frames:
14        raise ValueError("No data loaded. Load data first.")
15
16    # Get class distribution
17    class_counts = {cls: len(frames) for cls, frames in self.frames.items()}
18    print(f"Original class distribution: {class_counts}")
19
20    if method == 'undersample':
21        self._undersample_frames(random_state)
22    elif method == 'oversample':
23        self._oversample_frames(random_state)

```

Explanation: Class imbalance is a common problem in machine learning. This method provides two strategies:

- **Undersampling:** Reduces samples from majority classes to match the minority class count
- **Oversampling:** Increases samples from minority classes using bootstrapping

Why balance at frame level?

- Prevents bias in feature extraction
- Ensures equal representation during model training
- More effective than balancing after feature extraction

1.4.2 Undersampling Implementation

```

1 def _undersample_frames(self, random_state=None):
2     """Undersample frames to match the minority class"""
3     if random_state is not None:
4         random.seed(random_state)
5         np.random.seed(random_state)
6
7     # Find the minority class count
8     class_counts = [len(frames) for frames in self.frames.values()]
9     minority_count = min(class_counts)
10
11    # Undersample each class
12    balanced_frames = {}
13    for class_name, frames in self.frames.items():
14        if len(frames) > minority_count:
15            # Undersample

```

```

16         balanced_frames[class_name] = random.sample(frames, minority_count)
17     else:
18         # Keep all samples
19         balanced_frames[class_name] = frames
20
21     self.frames = balanced_frames

```

Explanation: Undersampling works by:

1. **Finding the smallest class** (minority count)
2. **For each class** with more samples than the minority:
 - Randomly select a subset of size `minority_count`
 - Uses `random.sample()` for non-replacement sampling
3. **Preserves all samples** from classes with fewer samples

Advantages:

- Faster training (fewer samples)
- Reduces computational requirements
- Prevents model from being biased toward majority classes

1.4.3 Oversampling Implementation

```

1 def _oversample_frames(self, random_state=None):
2     """Oversample frames to match the majority class"""
3     if random_state is not None:
4         random.seed(random_state)
5         np.random.seed(random_state)
6
7     # Find the majority class count
8     class_counts = [len(frames) for frames in self.frames.values()]
9     majority_count = max(class_counts)
10
11     # Oversample each class
12     balanced_frames = {}
13     for class_name, frames in self.frames.items():
14         if len(frames) < majority_count:
15             # Oversample with replacement
16             balanced_frames[class_name] = resample(
17                 frames,
18                 replace=True,
19                 n_samples=majority_count,
20                 random_state=random_state
21             )
22         else:
23             # Keep all samples
24             balanced_frames[class_name] = frames
25
26     self.frames = balanced_frames

```

Explanation: Oversampling works by:

1. **Finding the largest class** (majority count)
2. **For each class** with fewer samples than the majority:
 - Uses `resample()` with replacement to create new samples
 - Bootstraps existing samples to reach `majority_count`
3. **Preserves all samples** from classes with sufficient samples

Advantages:

- Preserves all original data
- Better for small datasets
- Can improve model performance on minority classes

1.5 Data Access Methods

```
1 def get_data_dict(self):
2     """Return the frames dictionary"""
3     return self.frames.copy()
4
5 def get_class_names(self):
6     """Return the list of class names"""
7     return self.class_names.copy()
8
9 def get_total_samples(self):
10    """Return total number of samples across all classes"""
11    return sum(len(frames) for frames in self.frames.values())
```

Explanation: These accessor methods provide controlled access to the internal data:

- `get_data_dict()`: Returns a copy of the frames dictionary to prevent accidental modification
- `get_class_names()`: Provides the list of unique class labels
- `get_total_samples()`: Calculates the total dataset size

Design Philosophy:

- **Encapsulation:** Internal data structures are protected
- **Immutability:** Returns copies to prevent external modifications
- **Convenience:** Easy access to commonly needed information

Chapter 2

Data Loading Utilities (LovdogData.py)

The **LovdogData** module provides specialized loading functions for different data formats, particularly focused on 2D array data commonly used in image processing and computer vision.

2.1 Main Loading Function

```
1 def load_file_with_extra(file_path, extra_info=None):
2     if extra_info is None:
3         raise ValueError("extra_info must be provided")
4
5     if 'type' not in extra_info.keys():
6         raise ValueError("type must be provided in extra_info")
7
8     if extra_info['type'] == '2d':
9         return load_file_with_extra_2d(file_path, extra_info)
10    elif extra_info['type'].lower() == '2drow':
11        return load_file_with_extra_2drow(file_path, extra_info)
12    else:
13        raise ValueError("Valid types are '2d', '2drow', '1drow (not implemented yet)')")
```

Explanation: This function serves as a dispatcher that routes to the appropriate loading function based on the data type specified in `extra_info`:

- '2d': For files containing a single 2D array
- '2drow': For files where each line represents a flattened 2D array
- '1drow': Reserved for future 1D data implementations

Key Features:

- **Type safety:** Validates that `extra_info` contains required parameters
- **Extensibility:** Easy to add new data format handlers
- **Error handling:** Clear error messages for missing or invalid parameters

2.2 2D Array Loading

```
1 def load_file_with_extra_2d(file_path, extra_info=None):
2     """
3     Loads a file containing a single 2D array.
4     """
5     if extra_info is None:
6         extra_info = {}
7
8     file_path = Path(file_path)
9     if not file_path.exists():
10        print(f"Error: File not found: {file_path}")
11        return []
12
13    # Get the essential shape information
14    try:
```

```

15     cols = int(extra_info['cols'])
16     rows = int(extra_info['rows'])
17 except KeyError:
18     print(f"Error: 'cols' and 'rows' must be provided in extra_info for file {file_path}.")
19     return []
20
21 frames_per_file = []
22 delimiter = extra_info.get('delimiter', None)
23
24 try:
25     frame_2d = np.loadtxt(file_path, delimiter=delimiter).reshape((rows, cols))
26     frames_per_file.append(frame_2d)
27
28     print(f"DEBUG: Loaded {len(frames_per_file)} frames from {file_path}")
29     return frames_per_file

```

Explanation: This function loads files that contain a single 2D array:

1. **File validation:** Checks if the file exists
2. **Parameter extraction:** Gets dimensions from `extra_info`
3. **Data loading:** Uses `np.loadtxt()` to read the file
4. **Reshaping:** Converts flattened data to 2D using specified dimensions
5. **Return format:** Always returns a list (for consistency)

Typical Use Cases:

- Single images stored as text files
- Pre-computed feature matrices
- 2D sensor data readings

2.3 2D Row-wise Loading

```

1 def load_file_with_extra_2dRow(file_path, extra_info=None):
2     """
3     Loads a file where each line is a flattened 1D array representing a single frame.
4     """
5     # [Parameter validation similar to above]
6
7     try:
8         with open(file_path, 'r') as f:
9             for line_num, line in enumerate(f):
10                 # Skip empty lines
11                 if not line.strip():
12                     continue
13
14                 # Convert the line of text into a list of numbers
15                 data_1d = np.loadtxt([line], delimiter=delimiter)
16
17                 # Check if the line has the correct number of elements
18                 expected_elements = cols * rows
19                 if data_1d.size != expected_elements:
20                     print(f"Warning: Line {line_num+1} has {data_1d.size} elements, "
21                           f"but {expected_elements} are needed for a {rows}x{cols} frame. Skipping line.")
22                     continue
23
24                 # Reshape the 1D array into a 2D frame
25                 frame_2d = data_1d.reshape((rows, cols))
26                 frames_per_file.append(frame_2d)

```

Explanation: This function handles files where each line represents a separate 2D array:

1. **Line-by-line processing:** Reads each line independently
2. **Data validation:** Checks if each line has the correct number of elements
3. **Individual reshaping:** Converts each line to a 2D array

4. **Robust error handling:** Skips invalid lines but continues processing

Typical Use Cases:

- Time series of 2D data (video frames, sensor sequences)
- Multiple observations in a single file
- Batch processing of related 2D arrays

Key Differences from load_file_with_extra_2d:

- Returns **multiple frames** (one per valid line)
- Handles **mixed valid/invalid** data gracefully
- Suitable for **sequential data** processing

2.4 Data Augmentation

```

1 def translate_px_aug(px, extra_info):
2     if extra_info is None:
3         extra_info = {}
4
5     offset_x = extra_info.get('offset_x', 1)
6     offset_y = extra_info.get('offset_y', 1)
7     tremble = extra_info.get('tremble', True)
8
9     px_new = px.copy() # Start with a copy of the original list
10
11     for p in px:
12         # Translate the pixels positively
13         px_mod = np.roll(p, shift=offset_y, axis=0)
14         px_mod = np.roll(px_mod, shift=offset_x, axis=1)
15         np.append(px_new, px_mod)
16
17     if tremble:
18         for p in px:
19             # Translate the pixels negatively
20             px_mod = np.roll(p, shift=-offset_y, axis=0)
21             px_mod = np.roll(px_mod, shift=-offset_x, axis=1)
22             np.append(px_new, px_mod)
23
24     return np.array(px_new)

```

Explanation: This function provides data augmentation through pixel translation:

- **Positive translation:** Shifts pixels right/down using `np.roll()`
- **Negative translation:** Shifts pixels left/up (when `tremble=True`)
- **Circular boundary handling:** `np.roll()` wraps around edges
- **Parameter control:** Translation amounts configurable via `extra_info`

Augmentation Benefits:

- **Increased dataset size** without collecting new data
- **Improved model robustness** to spatial variations
- **Reduced overfitting** by presenting varied examples

Use with LovdogDataFrames:

```

1 # Apply augmentation to loaded data
2 data_loader.augmentation_data(translate_px_aug,
3                               {'offset_x': 2, 'offset_y': 2, 'tremble': True})

```


Chapter 3

Feature Extraction (LovdogLBP.py)

The **LBPFeatures** class implements Local Binary Pattern feature extraction, a powerful texture analysis technique commonly used in image processing and computer vision applications.

3.1 Main Class: LBPFeatures

```
1 class LBPFeatures:
2     features = [
3         "File",
4         "mean",
5         "variance",
6         "correlation",
7         "contrast",
8         "homogeneity",
9         "class"
10    ]
11    verbose = 1
12
13    def __init__(self, data_loader=None):
14        self.data_loader = data_loader
15        self.LBP = {}
16        self.features_table = pd.DataFrame(columns=self.features)
17
18        # LBP parameters with defaults
19        self.ratio = 1
20        self.points = 8
21        self.method = "uniform"
22        self.window_size = [3]
```

Explanation: The LBPFeatures class extracts texture features using the Local Binary Pattern algorithm:

- **features:** Defines the output feature columns (statistical measures)
- **data_loader:** Reference to a LovdogDataFrames instance for data access
- **LBP:** Stores intermediate LBP images for each class
- **features_table:** Final DataFrame containing extracted features

LBP Parameters:

- **points:** Number of circularly symmetric neighbor points (typically 8, 16, or 24)
- **ratio:** Radius of the circle (scaled by points)
- **method:** LBP variant ('default', 'ror', 'uniform', 'var')
- **window_size:** For sliding window analysis of LBP images

3.2 LBP Computation

```

1 def compute_lbp_from_data_loader(self):
2     """
3     Compute LBP features using data from the LovdogDataFrames object.
4     """
5     if self.data_loader is None:
6         raise ValueError("No data loader provided. Call set_data_loader() first.")
7
8     frames_dict = self.data_loader.get_data_dict()
9     class_names = self.data_loader.get_class_names()
10
11     for class_name in class_names:
12         print(f" Processing class: {class_name}")
13         self.LBP[class_name] = []
14
15         for i, frame in enumerate(frames_dict[class_name]):
16             # Convert frame to uint8 if needed
17             if isinstance(frame, np.ndarray):
18                 # Normalize and convert to uint8 for LBP processing
19                 frame_normalized = cv.normalize(frame, None, 0, 255, cv.NORM_MINMAX).astype(np.uint8)
20                 lbp = local_binary_pattern(frame_normalized, self.points, self.ratio, self.method).astype(np.
uint8)
21
22                 # Use a generic filename or index
23                 file_id = f"{class_name}_{i:04d}"
24                 self.LBP[class_name].append({"File": file_id, "LBP": lbp, "class": class_name})

```

Explanation: This method processes data from the data loader through the LBP pipeline:

1. **Data access:** Retrieves frames organized by class from the data loader
2. **Normalization:** Converts input data to 8-bit format required by LBP
3. **LBP computation:** Applies the Local Binary Pattern algorithm
4. **Result storage:** Stores LBP images with metadata for later feature extraction

LBP Algorithm Overview:

- **Local analysis:** Examines each pixel relative to its neighbors
- **Binary patterns:** Creates binary codes representing local texture
- **Rotation invariance:** 'uniform' method provides rotation-invariant features
- **Texture encoding:** Converts local texture patterns into numerical codes

3.3 Feature Extraction from LBP

```

1 def compute_features_from_lbp(self):
2     """Compute features from LBP images (works with both old and new data sources)"""
3     bin_total = 25
4     bin_value = 0
5     for c in self.LBP.keys():
6         print(f"Processing {c} images...")
7         fd_size = len(self.LBP[c])
8         bin_value = fd_size // bin_total
9         fd_count = 0
10        for i in self.LBP[c]:
11            for ws in self.window_size:
12                result = self.lbp_window_analysis(i, ws, _class=c)
13                self.features_table = LBPFeatures.append_features_row(result, self.features_table)
14            if fd_count % bin_value == 0:
15                print("*", end="", flush=True)
16                fd_count += 1

```

Explanation: This method converts LBP images into statistical features:

- **Progress tracking:** Visual feedback during processing
- **Window analysis:** Applies sliding window to extract local features

- **Multiple scales:** Processes each window size specified
- **Feature aggregation:** Combines results into final feature table

Progress Indicators:

- `bin_total`: Number of progress markers to display
- `bin_value`: Calculates how many processed per marker
- Visual feedback helps monitor long-running feature extraction

3.4 Window-based Feature Analysis

```

1 @staticmethod
2 def lbp_window_analysis(lbp_img, window_size, _class=None):
3     x = 0
4     y = 0
5     lbp_image = lbp_img["LBP"]
6     lbp_file = lbp_img["File"]
7     analysis = []
8     run_boggle = [ "-", "\\ ", "|", "/" ]
9     idx_boggle = 0
10
11     if window_size < 0:
12         # Process entire image as single window
13         lbp_feat = LBPFeatures.get_features_from_window(lbp_image)
14         lbp_feat["File"] = lbp_file
15         if _class is not None:
16             lbp_feat["class"] = _class
17         analysis.append(lbp_feat)
18         return pd.DataFrame(analysis, columns=LBPFeatures.features)
19
20     # Sliding window approach
21     for x in range(lbp_image.shape[0] - window_size):
22         for y in range(lbp_image.shape[1] - window_size):
23             window = lbp_image[x:x + window_size, y:y + window_size]
24             lbp_feat = LBPFeatures.get_features_from_window(window)
25             lbp_feat["File"] = lbp_file
26             if _class is not None:
27                 lbp_feat["class"] = _class
28             analysis.append(lbp_feat)
29             # Progress animation
30             print(run_boggle[idx_boggle], end="", flush=True)
31             idx_boggle = idx_boggle + 1 if idx_boggle < 3 else 0
32             print("\b", end="", flush=True)

```

Explanation: This method implements sliding window analysis of LBP images:

- **Two modes:** Whole-image analysis (`window_size < 0`) or sliding window
- **Sliding window:** Extracts features from overlapping sub-regions
- **Progress animation:** Rotating cursor shows processing activity
- **Feature aggregation:** Collects features from all window positions

Window Analysis Benefits:

- **Spatial information:** Captures local texture variations
- **Multi-scale analysis:** Different window sizes capture different texture scales
- **Robust features:** Averages out local noise through multiple observations

3.5 GLCM Feature Extraction

```

1 @staticmethod
2 def get_features_from_window(window):
3     features = {}
4     glcm = graycomatrix(window, [1], [0], symmetric=True, normed=True, levels=59)
5     features["mean"] = graycoprops(glcm, 'mean')[0][0]
6     features["variance"] = graycoprops(glcm, 'variance')[0][0]
7     features["correlation"] = graycoprops(glcm, 'correlation')[0][0]
8     features["contrast"] = graycoprops(glcm, 'contrast')[0][0]
9     features["homogeneity"] = graycoprops(glcm, 'homogeneity')[0][0]
10    return features

```

Explanation: This static method computes Gray-Level Co-occurrence Matrix (GLCM) features from LBP windows:

- **GLCM computation:** Creates a matrix showing how pixel pairs co-occur
- **Parameter choices:**
 - distance=1: Neighboring pixels
 - angle=0: Horizontal direction
 - symmetric=True: Reduces computation and provides rotational invariance
 - levels=59: Quantization levels for LBP values (8-bit range)
- **Statistical features:** Five key texture measures

GLCM Feature Definitions:

- **Mean:** Average gray level intensity
- **Variance:** Spread of gray level distribution
- **Correlation:** Linear dependency of gray levels
- **Contrast:** Local intensity variations (edge strength)
- **Homogeneity:** Closeness of element distribution to GLCM diagonal

3.6 Machine Learning Data Preparation

```

1 def get_ml_data(self, include_file=False):
2     """
3     Return features and labels in standard scikit-learn format.
4
5     Args:
6         include_file (bool): Whether to include the 'File' column in features
7
8     Returns:
9         tuple: (X_features, y_labels) where:
10             X_features: numpy array or DataFrame of features
11             y_labels: numpy array or Series of class labels
12     """
13     if include_file:
14         X = self.features_table.drop('class', axis=1)
15     else:
16         X = self.features_table.drop(['File', 'class'], axis=1)
17
18     y = self.features_table['class']
19     return X, y

```

Explanation: This method converts the feature table into standard machine learning format:

- include_file=False: Standard case, returns only numerical features
- include_file=True: Includes filename for debugging or tracking
- **X:** Feature matrix (samples × features)

- **y**: Label vector (samples $\times$ 1)

Compatibility:

- Direct input to scikit-learn classifiers
- Compatible with train/test splitting
- Works with cross-validation and grid search

Chapter 4

Model Selection (LovdogMS.py)

The **ModelSelection** class is the core of the automated machine learning framework, providing comprehensive model evaluation, hyperparameter tuning, and performance analysis.

4.1 Main Class: ModelSelection

```
1 class ModelSelection:
2     def __init__(
3         self,
4         name: str,
5         # Multiple data input options for flexibility
6         csv_path: Optional[str] = None,
7         features_dataframe: Optional[pd.DataFrame] = None,
8         ml_data_tuple: Optional[Tuple] = None,
9         # Dataset configuration
10        target_column: str = "class",
11        test_size: float = 0.2,
12        random_state: Optional[int] = None,
13        # Performance optimization
14        best_only_mode: bool = False,
15        # Output configuration
16        plots_output_directory: str = "plots",
17        # Classifier configuration
18        classifiers_dict: Optional[Dict[str, Any]] = None,
19        grid_parameters_dict: Optional[Dict[str, Any]] = None
20    ):
```

Explanation: The ModelSelection class provides a unified interface for automated model evaluation:

- **name:** Unique identifier for tracking multiple experiments
- **Multiple data inputs:** CSV files, DataFrames, or direct (X,y) tuples
- **target_column:** Specifies which column contains class labels
- **test_size:** Train/test split ratio (0.2 = 80% train, 20% test)
- **best_only_mode:** Optimizes performance by only analyzing the best model
- **plots_output_directory:** Where to save diagnostic plots

Data Input Flexibility:

- **csv_path:** Direct file loading for quick experiments
- **features_dataframe:** Pre-loaded pandas DataFrame
- **ml_data_tuple:** Direct (X,y) input from LBP feature extraction

4.2 Default Classifier Configuration

```

1 default_classifiers = {
2     'SVC': SVC(),
3     'KNeighbors': KNeighborsClassifier(),
4     'RandomForest': RandomForestClassifier(),
5     'LogisticRegression': LogisticRegression(max_iter=850),
6 }
7
8 default_grid_parameters = {
9     'SVC': {
10         'clf__C': [0.001, 0.01, 0.1, 1, 10, 20],
11         'clf__kernel': ['poly', 'sigmoid', 'rbf'],
12         'clf__gamma': ['scale', 'auto'],
13     },
14     'RandomForest': {
15         'clf__min_samples_split': [2, 4, 8, 16],
16         'clf__n_estimators': [72, 144, 288, 576],
17         'clf__max_depth': [3, 9, 27],
18     }
19 }

```

Explanation: The framework includes four well-established classifiers with optimized parameter grids:

- **Support Vector Classifier (SVC):** Effective for high-dimensional data
- **K-Nearest Neighbors:** Simple, interpretable, good for small datasets
- **Random Forest:** Robust, handles non-linear relationships well
- **Logistic Regression:** Fast, good baseline, probabilistic outputs

Parameter Grid Design:

- **Exponential scales:** For regularization parameters (0.001 to 20)
- **Power-of-two progressions:** For ensemble sizes (72 to 576)
- **Multiple strategies:** Different kernel types, splitting criteria
- **Computational efficiency:** Balanced between exploration and computation time

4.3 Core Model Selection Process

```

1 def selectModels(self):
2     """Run grid search for all classifiers, including AUC calculation"""
3     self.results = {}
4     all_metrics = []
5
6     for tag, algorithm in self.classifiers.items():
7         print(f"Training {tag}...")
8
9         # Prepare pipeline
10        pipe = Pipeline([
11            ('scaler', StandardScaler()),
12            ('clf', algorithm)
13        ])
14
15        # SetUp grid
16        modelsGrid = GridSearchCV(
17            pipe,
18            param_grid=self.gridParameters[tag],
19            scoring={
20                "accuracy": "accuracy",
21                "precision": "precision_weighted",
22                "recall": "recall_weighted",
23                "f1": "f1_weighted"
24            },
25            refit="accuracy",
26            cv=5,
27            n_jobs=-1,
28            verbose=1
29        )

```

Explanation: The main model selection method implements a comprehensive evaluation pipeline:

1. **Pipeline construction:** StandardScaler + Classifier for consistent preprocessing
2. **GridSearchCV setup:** Exhaustive hyperparameter search with cross-validation
3. **Multi-metric evaluation:** Accuracy, Precision, Recall, F1-score
4. **Parallel processing:** `n_jobs=-1` uses all available CPU cores
5. **5-fold cross-validation:** Robust performance estimation

Pipeline Benefits:

- **Automatic scaling:** Features standardized before classification
- **Prevention of data leakage:** Scaling fitted only on training folds
- **Consistent preprocessing:** Same transformation for all models

4.4 Comprehensive Performance Metrics

```

1 # Calculate all basic metrics
2 accuracy = accuracy_score(self.test_data['y'], y_pred)
3 precision = precision_score(self.test_data['y'], y_pred, average='weighted')
4 recall = recall_score(self.test_data['y'], y_pred, average='weighted')
5 f1 = f1_score(self.test_data['y'], y_pred, average='weighted')
6
7 # Calculate AUC if the model supports probability predictions
8 auc_score = None
9 if hasattr(best_model, 'predict_proba'):
10     try:
11         y_pred_proba = best_model.predict_proba(self.test_data['x'])
12
13         # For multi-class classification, use One-vs-Rest approach
14         if len(np.unique(self.test_data['y'])) > 2:
15             y_test_bin = label_binarize(self.test_data['y'],
16                                         classes=np.unique(self.test_data['y']))
17             # Calculate micro-average AUC
18             fpr, tpr, _ = roc_curve(y_test_bin.ravel(), y_pred_proba.ravel())
19             auc_score = auc(fpr, tpr)
20         else:
21             # For binary classification
22             fpr, tpr, _ = roc_curve(self.test_data['y'], y_pred_proba[:, 1])
23             auc_score = auc(fpr, tpr)

```

Explanation: The framework computes multiple performance metrics for comprehensive model evaluation:

- **Accuracy:** Overall correct classification rate
- **Precision:** Ratio of true positives to predicted positives
- **Recall:** Ratio of true positives to actual positives
- **F1-score:** Harmonic mean of precision and recall
- **AUC-ROC:** Area Under ROC Curve for probabilistic models

Metric Selection Rationale:

- **Weighted averages:** Handle multi-class and imbalanced datasets
- **AUC calculation:** Only for models supporting probability predictions
- **Micro-averaging:** For multi-class AUC computation
- **Comprehensive view:** Different metrics highlight different strengths

4.5 Best Model Selection

```

1 # Determine the best classifier (considering AUC if available)
2 if not self.metrics_df.empty:
3     # Prefer classifiers that have AUC scores for ranking
4     has_auc_df = self.metrics_df[self.metrics_df['AUC_Score'] != 'N/A'].copy()
5     if not has_auc_df.empty:
6         # Convert AUC_Score to float for sorting
7         has_auc_df['AUC_Score'] = has_auc_df['AUC_Score'].astype(float)
8         best_idx = has_auc_df['AUC_Score'].idxmax()
9         self.best_model_tag = has_auc_df.loc[best_idx, 'Classifier']
10    else:
11        # Fallback to accuracy if no AUC available
12        best_idx = self.metrics_df['Accuracy'].idxmax()
13        self.best_model_tag = self.metrics_df.loc[best_idx, 'Classifier']
14
15    self.best_model = self.results[self.best_model_tag]['best_estimator']

```

Explanation: The framework implements a smart best-model selection strategy:

1. **Priority to AUC:** If available, AUC is preferred for model ranking
2. **Fallback to accuracy:** For models without probability outputs
3. **Automatic type conversion:** Handles string 'N/A' values gracefully
4. **Model retrieval:** Stores both the best model and its metadata

Why Prefer AUC?

- **Threshold-independent:** Not affected by classification threshold choice
- **Probability quality:** Measures how well probabilities are calibrated
- **Robust to imbalance:** Works well with unbalanced class distributions

4.6 Diagnostic Visualization

```

1 def generatePlots(self):
2     """Generate plots - only for best classifier if best_only=True, otherwise for all"""
3     if self.best_only_mode:
4         self._generatePlotsForBest()
5     else:
6         self._generatePlotsForAll()
7
8 def _generateLearningCurve(self, tag):
9     """Generate learning curve for a specific classifier"""
10    model = self.results[tag]['best_estimator']
11
12    train_sizes, train_scores, test_scores = learning_curve(
13        model, self.data, self.target, cv=5, n_jobs=-1,
14        train_sizes=np.linspace(0.1, 1.0, 5),
15        scoring='accuracy'
16    )

```

Explanation: The framework generates three types of diagnostic plots:

- **Learning Curves:** Show how performance changes with training set size
- **ROC Curves:** Display trade-off between true positive and false positive rates
- **Confusion Matrices:** Detailed breakdown of classification errors

Plot Generation Modes:

- `best_only_mode=True`: Only generate plots for the best-performing model
- `best_only_mode=False`: Generate plots for all evaluated models

Learning Curve Interpretation:

- **Large gap:** Indicates overfitting (model too complex)
- **Convergence:** Both curves plateau at similar levels (good fit)
- **Poor performance:** Both curves remain low (underfitting)

4.7 Model Persistence and Export

```

1 def saveModels(self, path):
2     """Save all trained models to disk"""
3     os.makedirs(path, exist_ok=True)
4     for tag, result in self.results.items():
5         joblib.dump(result['best_estimator'], f"{path}/{tag}.joblib")
6
7 def getMetricsDataFrame(self):
8     """Create a DataFrame with all metrics including AUC for CSV export"""
9     if self.metrics_df.empty:
10        # If selectModels hasn't been run, create basic metrics dataframe
11        metrics_data = []
12        for tag, result in self.results.items():
13            metrics_data.append({
14                'Classifier': tag,
15                'Accuracy': result['test_accuracy'],
16                'Precision': result['test_precision'],
17                'Recall': result['test_recall'],
18                'F1_Score': result['test_f1'],
19                'AUC_Score': result['test_auc'] if result['test_auc'] is not None else 'N/A',
20                'CV_Score': result['cv_best_score'],
21                'Best_Params': str(result['best_params']),
22                'Supports_Probability': result['supports_proba']
23            })
24
25        self.metrics_df = pd.DataFrame(metrics_data)
26
27    return self.metrics_df

```

Explanation: The framework provides comprehensive model and results export:

- **Model serialization:** All trained models saved using joblib
- **Directory creation:** Automatic creation of output directories
- **Comprehensive metrics:** Detailed performance statistics in DataFrame format
- **CSV export ready:** Direct compatibility with spreadsheet software
- **Parameter documentation:** Best hyperparameters included for reproducibility

Export Benefits:

- **Reproducibility:** All models and parameters saved
- **Analysis ready:** Metrics formatted for further statistical analysis
- **Deployment ready:** Trained models can be loaded for inference
- **Documentation:** Complete record of experimental results